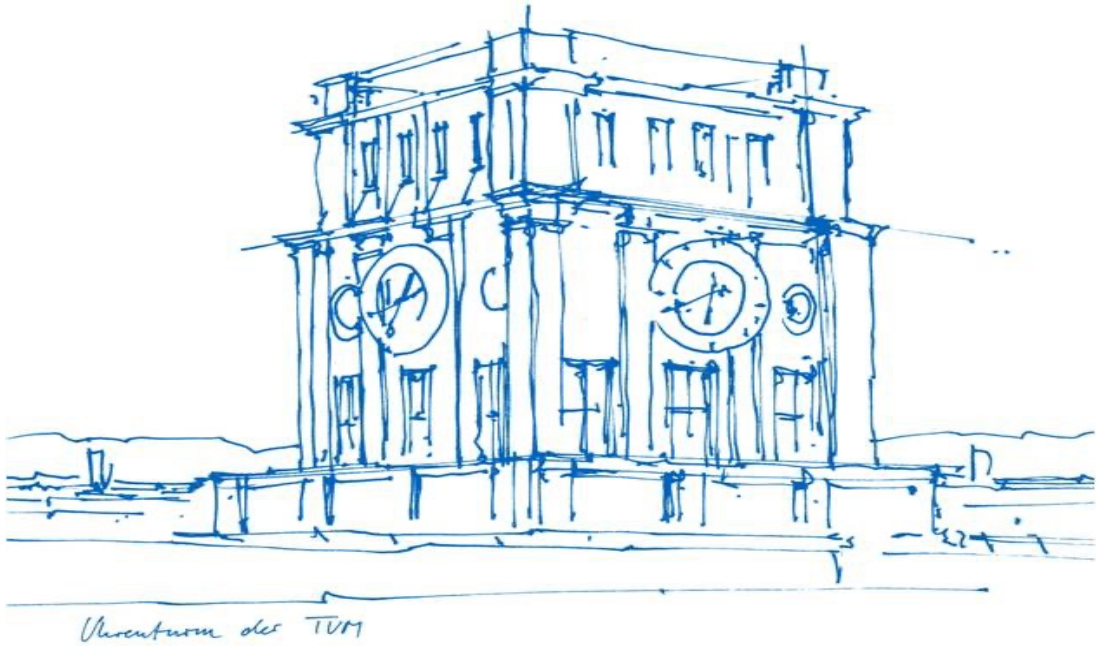
IN2178 – Security Engineering
Exercise 6 – Hardware-Based Protection

Patrick Stöckle

Informatics IV – Chair of Software and Systems Engineering

Summer 2022

30.06.2022



Exercise 6.1: Hardware-Based Security Concepts

1. Explain the main difference between dynamic and static security in hardware-based protections.

Exercise 6.1: Hardware-Based Security Concepts

1. Explain the main difference between dynamic and static security in hardware-based protections.

Answer

- **Static** protection in which system software components (BIOS, boot loader, OS, etc) constitute a hash chain and thereby protected (if the system starts secure; it stays secure!)
- **Dynamic** protection in which a CPU-level security enforcement protects programs through execution (System can start right, but things could go wrong at runtime)
- Dynamic protections guarantee **static security** by verifying the signature of programs upon load

Exercise 6.1: Hardware-Based Security Concepts

2. Name and describe three types of attestations.

Exercise 6.1: Hardware-Based Security Concepts

2. Name and describe three types of attestations.

Answer

Attestation is a mechanism for software to prove its identity. The goal of attestation is to prove to a remote party that your operating system and application software are intact and trustworthy. Attestation can be done locally between two programs or remotely between a program and a remote verifier.

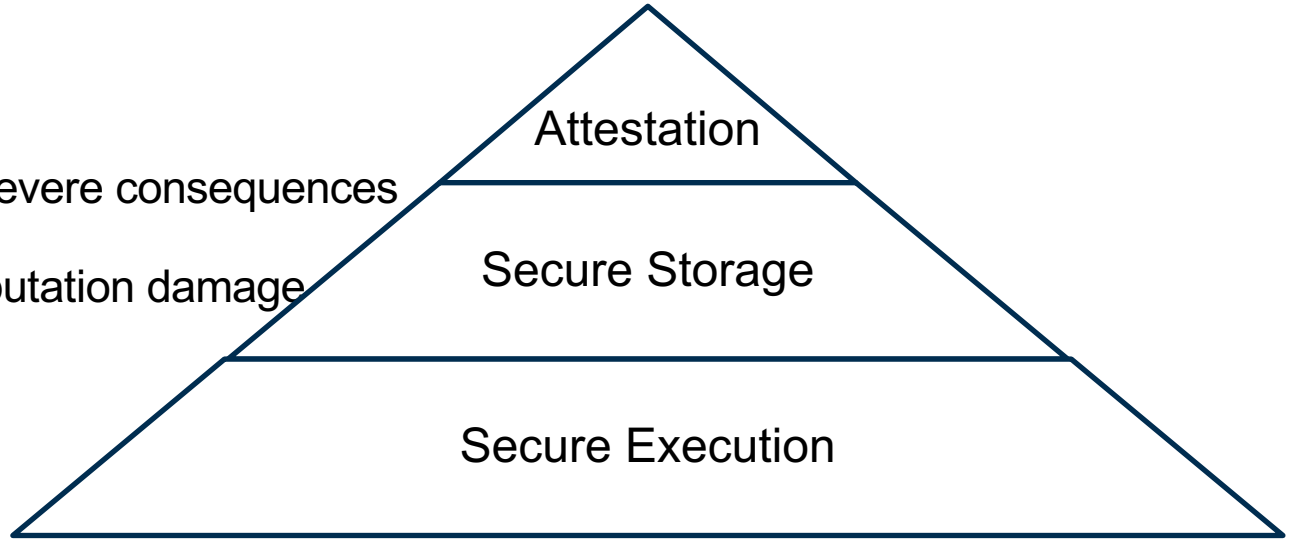
- Identification of devices
  - An IT organization needs to be certain that only enterprise-owned machines are allowed on the enterprise's network (IP and MAC address are not security identifiers)
- Device health attestation
  - An IT organization needs to frequently check enterprise machines for possible compromises
- Secure storage of keys
  - Enterprise sensitive keys need to be secured from other software on the system

Exercise 6.1: Hardware-Based Security Concepts

3. Briefly explain two use cases in which hardware-based security can be useful.

Hardware-Based Protection: The Protection Pyramid

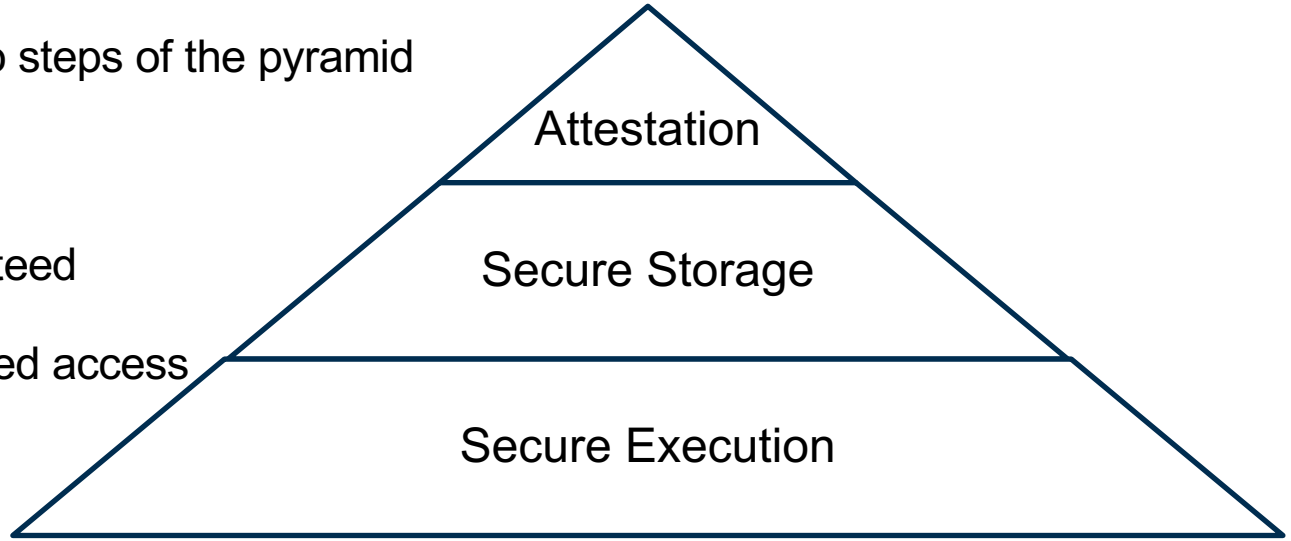
- Secure Execution mitigates the following threats
  - All the servers on which the banking software stack is being executed are exposed to IT users
  - Malicious insiders may manipulate system's configurations and services
- Secure Storage
  - Disclosure of to the accounts' credentials can have severe consequences
  - Leakage of the account funds can lead to serious reputation damage



- Attestation
  - The system shall be able to provide unforgeable proofs of its integrity state at the granularity of the services and configurations to local or remote services

Mapping Banking System Threats to the Protection Pyramid

- Secure Execution
  - The execution of programs/services shall be protected against manipulations
  - Genuine cryptographic functions shall be provided for programs/services
  - Secure execution provides the basis for the other two steps of the pyramid
- Secure Storage
  - The confidentiality of system secrets shall be guaranteed
  - Only the program that owns the secret shall be granted access
- Attestation
  - Worst yet, attacks may go unnoticed for years!



Exercise 6.1: Hardware-Based Security Concepts

4. What are the limitations of TPM-based protections? Name and explain three limitations.

Trusted Platform Module (TPM) – Limitations

- We have to know and maintain all hashes for all versions of software!
  - Software patches and updates rolled out almost on a daily basis
- A third party verifier (read single point of failure) has to attest the system
- Starting secure (hash chain) does not necessarily guarantee a sustainable security
  - Simply put, securing binary representation does not guarantee secure execution
  - Exploit vulnerabilities? In mem? In exec?
- Not all computers have built-in TPM chips
- Either protect the entire system all the way from the boot or nothing!

Exercise 6.1: Hardware-Based Security Concepts

5. Can hardware-based protection render software-based protection obsolete? Justify your answer with example(s).

Exercise 6.1: Hardware-Based Security Concepts

5. Can hardware-based protection render software-based protection obsolete? Justify your answer with example(s).

Answer (Opinion)

Existing hardware-based protections (i.e., enclave-based protection) probably will **not** render software-based protection obsolete.

Why?

- Such hardware is not available in a majority of computing devices (particularly IoT devices)
- As long as confidentiality and integrity remain relevant requirements in computing devices with no trusted hardware, software-based protection remains relevant.
- The existing trusted hardware does not fully address IP concerns such as code confidentiality. That is, obfuscation continues to be a relevant measure against reverse engineering

Exercise 6.1: Hardware-Based Security Concepts

5. Can hardware-based protection render software-based protection obsolete? Justify your answer with example(s).

Answer (Opinion)

Existing hardware-based protections (i.e., enclave-based protection) probably will **not** render software-based protection obsolete.

Why?

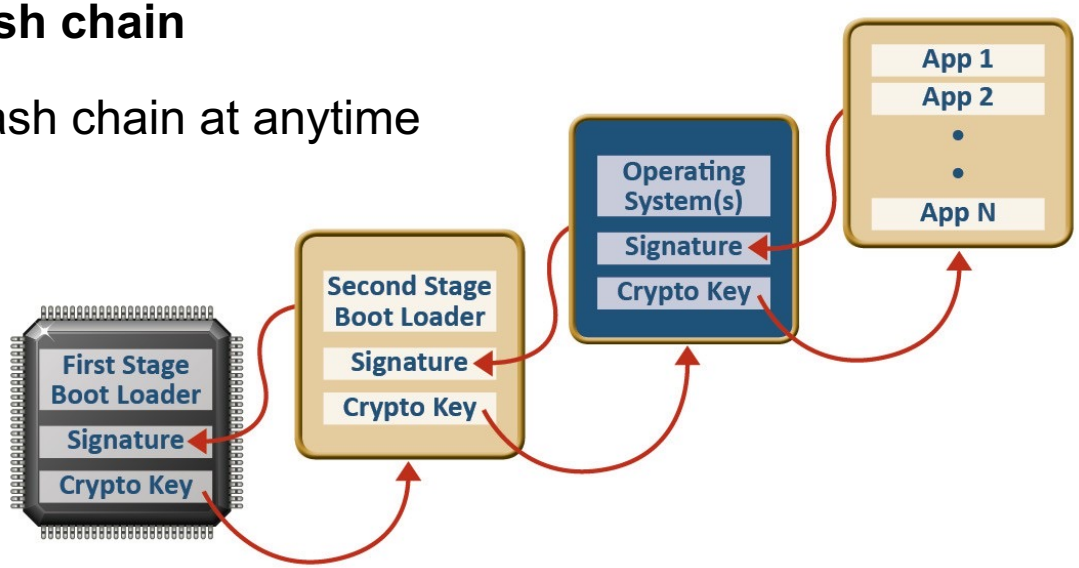
- Adversaries might be able to achieve their goals by manipulating program parts residing in the untrusted world instead of the enclave-based code. Since IO operations of programs cannot be placed inside enclaves, parts of programs have to reside in the untrusted world inevitably.
- In the current situation, hybrid approaches (combining software-based and hardware-based) appear to be viable options.

Exercise 6.1: Hardware-Based Security Concepts

6. Briefly explain how the hash chain mechanism protects a system from integrity violations.

Trusted Platform Module (TPM) – Attestation (aka Hash Chain)

- TPM comes with a trusted boot loader (first stage boot loader)
- TPM certifies the system boot loader (second stage boot loader)
- Boot loader certifies OS
- OS certifies Applications
- All certificates include the public key of the component that is certified along with a hash of the component binary file
- These certificates constitute a **hash chain**
- A remote verifier can attest the hash chain at anytime



Exercise 6.1: Hardware-Based Security Concepts

6. Briefly explain how the hash chain mechanism protects a system from integrity violations.

Answer

As shown in the previous slide, TPM constitutes a hash chain starting from a secure boot loader to the OS and applications. In the case of tampering attacks, the hashes will not match the expected values. Such mismatches can be verified by local application or remote attesters.

Exercise 6.1: Hardware-Based Security Concepts

7. How does SGX ensure that sealed data can exclusively be read by the enclave that owns the data?

Exercise 6.1: Hardware-Based Security Concepts

7. How does SGX ensure that sealed data can exclusively be read by the enclave that owns the data?

Answer

Sealing is a process in which an enclave securely persists its data to the disk. Encryption is performed using a private Seal Key that is unique to that particular platform and enclave, and is unknown to any other entity. During the manufacturing process, a unique set of keys are generated and stored in the processor's fuse array. There are certain measures to ensure that the key is only retrievable by the issuing enclave. Read more here: https://software.intel.com/content/www/us/en/develop/blogs/introduction-to-intel-sgx-sealing.html

Exercise 6.1: Hardware-Based Security Concepts

8. In exercise sheet 4, we have discussed attacks on SaaS, PaaS, and IaaS architectures. We assume that we are a tenant of such a SaaS cloud service by Mississippi Web Services (MWS). MWS claims that their machines now include TPM and SGX to improve the security of their virtual machines. Should we feel good about this? What attacks on IaaS/PaaS systems discussed before are now impossible?

Exercise 6.1: Hardware-Based Security Concepts

8. In exercise sheet 4, we have discussed attacks on SaaS, PaaS, and IaaS architectures. We assume that we are a tenant of such a SaaS cloud service by Mississippi Web Services (MWS). MWS claims that their machines now include TPM and SGX to improve the security of their virtual machines. Should we feel good about this? What attacks on IaaS/PaaS systems discussed before are now impossible?

Answer

Adequate utilization of security hardware can severely harden the system security. In principle, security hardware can facilitate the mitigation of threats to the confidentiality and integrity of assets. However, employing security hardware by itself does not necessarily guarantee security. The system has to undergo adequate security analysis to identify threats to its assets. Subsequently, relevant measures shall be implemented to protect system assets. In most cases, a combination of protections may be required to mitigate all the threats. To answer the question in a generic sense, TPM and SGX enable the cloud to provide means for attestation to their users.

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

In this assignment, we protect the given smart electricity meter application via enclave-based security.

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

The meter's counter shall not be modified by any other processes, but the meter program. In the lecture, you learned about enclave-based protection. Here, we use the Google Asylo framework for implementing enclave-based security. Asylo simplifies the development of enclave-based applications by providing a set of intuitive (high-level) functions for creating and using enclaves. Moreover, the Asylo framework is hardware agnostic, i.e., a program written in Asylo can be compiled to run on arbitrary secure hardware. As of now, Asylo supports both software simulated enclaves as well as Intel SGX. To switch between arbitrary hardware, users merely need to configure the backend hardware upon compilation. You can use the simulated enclave backend to solve this exercise.

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

1. Bring yourself up to speed with Asylo by following the getting started guide
https://asylo.dev/docs/guides/quickstart.html

Enclave-Based Security: A Technological Dilemma

- The developers are in charge of splitting their applications into trusted and untrusted partitions
- Different technologies exists for confidential computing: Open Enclave, Intel SGX, ARM TrustZone
- Every technology comes with its own SDK and documentation
  - Intel SGX's implementation differs from ARM TrustZone
- Switching between technologies often entails learning a whole set of development tools
- Codes written upon a particular SDK version may become obsolete over time (maintenance cost)
- Porting an existing enclave application to a different hardware is often expensive

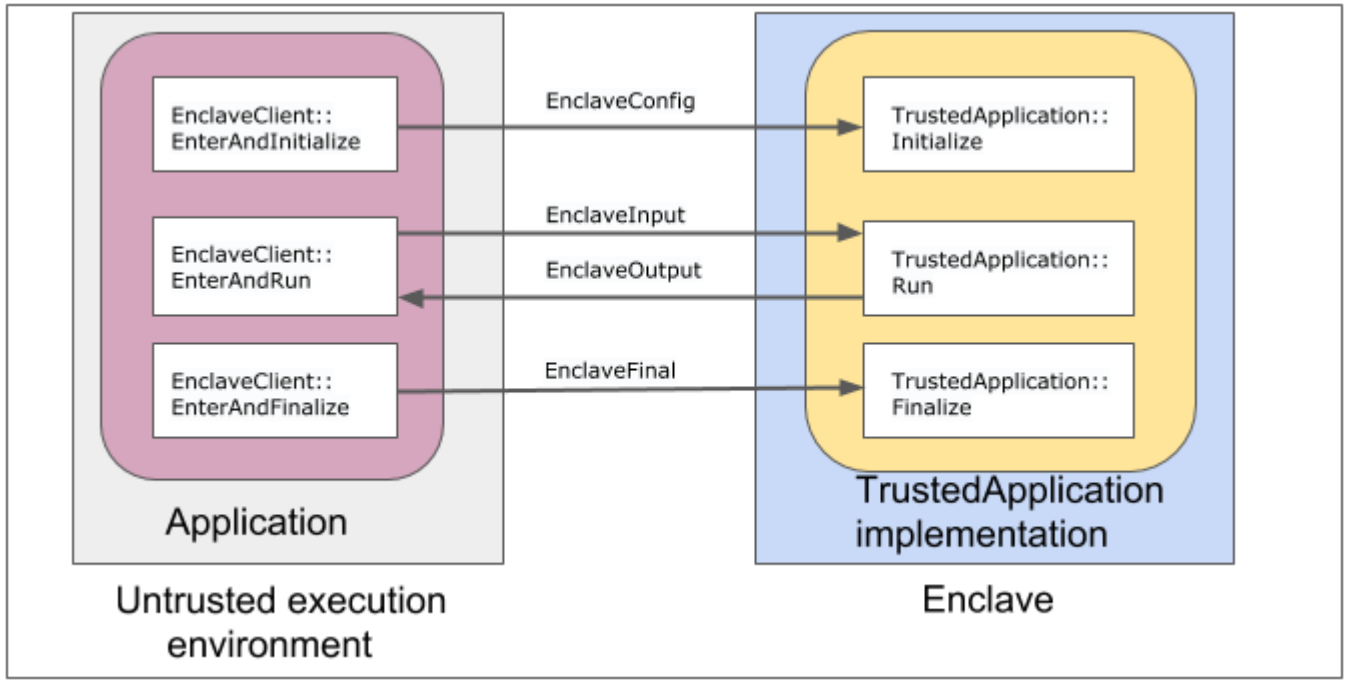
Enclave-Based Security with Asylo

- The developers are **still** in charge of splitting their applications into trusted and untrusted partitions
- The underlying technology (hardware) is abstracted away as backend

- Enclave programs are built via intuitive multi-purpose constructs
- A program can be compiled to various backends: Intel SGX or simulated enclave, others to come

- The program itself remains transparent to the underlying technology
  - Write one program, compile it to various (supported) backends
- Programs are written in a higher level of abstraction, so SDK changes do not propagate to user application usually

Enclave-Based Security with Asylo: Constructs cont'd



https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: Constructs

- **TrustedApplication** is the trusted component of an enclave application that is responsible for sensitive computations.
- **EnclaveClient** is the untrusted component of an enclave application that is responsible for communicating messages with the trusted component.
- **EnclaveManager** is a singleton object in the untrusted system that is responsible for managing enclave lifetimes and shared resources between enclaves.
- We refer to the process of switching from an untrusted execution environment to a trusted environment as *entering* an enclave. Every enclave provides a limited number of entry points where trusted execution may begin or resume.
- **The trusted environment** consists of one or more enclaves, which protect code and data in a sensitive workload. To create an enclave, we define a class which inherits from **TrustedApplication** and implement the logic to host in the enclave.
- **Untrusted environment** The untrusted API provides methods analogous to the enclave entry points that are defined by a TrustedApplication. These methods implement the necessary machinery to safely cross the enclave boundary. The inputs to these methods are extensible by the developer.

https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: Trusted Application

The TrustedApplication class declares methods corresponding to each entry point defined by the Asylo API.

- **Initialize** initializes the enclave with configuration values that are bound to the enclave's identity. This should be used for security-sensitive configuration settings.
- **Run** takes input messages from the untrusted environment code, performs trusted execution, and returns an output message to the untrusted environment.
- **Finalize** takes finalization values from the untrusted environment and is called before the enclave is destroyed.

If any of these methods are not overridden, they simply return an OkStatus.

https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: EnclaveClient

The EnclaveClient class is responsible for communicating messages with the trusted component.

- EnterAndInitialize passes enclave configuration settings and optional user-defined configuration extensions to the enclave. The configuration information is essential to the identity of the enclave.
- EnterAndRun passes an input message to the enclave, and receives an output message from the enclave. This method may be called an arbitrary number of times with different inputs after the enclave has been initialized.
- EnterAndFinalize passes enclave finalization data to the enclave.

Both EnterAndInitialize and EnterAndFinalize are private methods that are called by the friend class EnclaveManager.

https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: EnclaveManager

The EnclaveManager class is responsible for creating and managing enclave instances. This class is a friend class to EnclaveClient.

- **LoadEnclave** initializes the enclave with a call to EnterAndInitialize.
- **DestroyEnclave** destroys the enclave with a call to EnterAndFinalize.

Entering an enclave is analogous to making a system call. An enclave entry point implements a gateway to sensitive code with access to the enclave's resources. Arguments are copied into the enclave on entry and results are copied out on exit.

https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: Enclave Lifecycle

Part 1: Initialization

- The untrusted application performs a set of steps to initialize the trusted application
  - Instantiate **EnclaveManager**
  - Configure the options for fetching the enclave from the disk
  - Call **LoadEnclave** --- Implicitly invoke the enclave's **Initialize** method

Part 2: Secure Execution

- The untrusted application performs a set of steps to securely execute a workload in the trusted application
  - Provide input via the **EnclaveInput** message
  - Get a handle to the enclave via EnclaveManager::GetClient
  - Dispatch a call to the enclave, e.g., by calling **EnclaveClient::EnterAndRun**

Part 3: Finalization

- Destroy the enclave **EnclaveManager::DestroyEnclave**

- The Asylo framework will implicitly use the client to call the trusted application's **Finalize** method

https://asylo.dev/docs/concepts/api-overview.html

Enclave-Based Security with Asylo: Interaction model

In Asylo, enclaves operate on protocol-buffer messages; all enclave inputs and outputs are protocol buffers.

What are protocol buffers?

Protocol buffers are Google's language-neutral, platform-neutral, extensible mechanism for serializing structured data – think XML, but smaller, faster, and simpler. You define how you want your data to be structured once, then you can use special generated source code to easily write and read your structured data to and from a variety of data streams and using a variety of languages.

https://developers.google.com/protocol-buffers

https://asylo.dev/docs/concepts/api-overview.html

Protocol-Buffers: Defining A Message Type

You define your message type in a .proto file:

- Scalar and composite types are supported
- Each field in the definition has a **unique number**
- Fields can be **required**, **optional**, or **repeated**
  - Use **repeated** for arrays

```
message SearchRequest {
  required string query = 1;
  optional int32 page_number = 2;
  optional int32 result_per_page = 3;
}
```

https://developers.google.com/protocol-buffers/docs/overview

Protocol-Buffers: Extensions

Extensions let you declare that a range of field numbers in a message are available for third-party extensions. An extension is a placeholder for a field whose type is not defined by the original .proto file. This allows other .proto files to add to your message definition by defining the types of some or all of the fields with those field numbers.

```
message Foo {
  // ...
  extensions 100 to 199;
}
```

- This says that the range of field numbers [100, 199] in Foo is reserved for extensions. Other users can now add new fields to Foo in their own .proto files that import your .proto, using field numbers within your specified range – for example:

```
extend Foo {
  optional int32 bar = 126;
}
```

- This adds a field named bar with the field number 126 to the original definition of Foo.

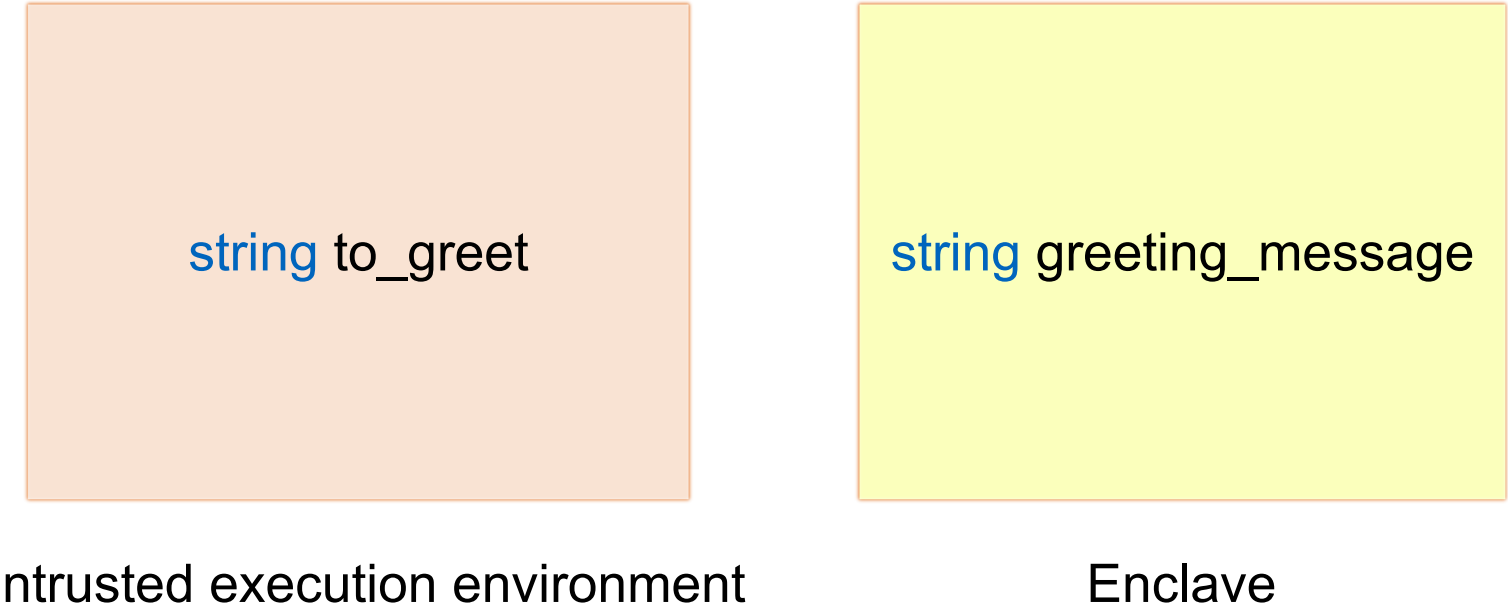
- Extensions use a specific interface:

```
Foo foo;
foo.SetExtension(bar, 15);
```

https://developers.google.com/protocol-buffers/docs/overview

Hello World in Asylo

Get users names (comma separated) from the input and generate a greeting message inside the enclave



https://github.com/google/asylo/tree/master/asylo/examples/hello_world

Hello World in Asylo: defining hello.proto

hello.proto

hello_world > hello.proto

```
syntax = "proto2";

package hello_world;

import "asylo/enclave.proto";

// Extension to EnclaveInput for use with HelloWorld enclave.
message HelloInput {
  optional string to_greet = 1;
}

message HelloOutput {
  optional string greeting_message = 1;
}

extend asylo.EnclaveInput {
  optional HelloInput enclave_input_hello = 8086;
}

extend asylo.EnclaveOutput {
  optional HelloOutput enclave_output_hello = 8087;
}
```

Hello World in Asylo: defining a hello world enclave

```
27  class HelloApplication : public asylo::TrustedApplication {
28   public:
29    HelloApplication() : visitor_count_(0) {}
30
31    asylo::Status Run(const asylo::EnclaveInput &input,
32                      asylo::EnclaveOutput *output) override {
33      if (!input.HasExtension(hello_world::enclave_input_hello)) {
34        return asylo::Status(asylo::error::GoogleError::INVALID_ARGUMENT,
35                             "Expected a HelloInput extension on input.");
36      }
37      std::string visitor =
38          input.GetExtension(hello_world::enclave_input_hello).to_greet();
39
40      LOG(INFO) << "Hello " << visitor;
41      if (output) {
42        LOG(INFO) << "Incrementing visitor count...";
43        output->MutableExtension(hello_world::enclave_output_hello)
44            ->set_greeting_message(
45                absl::StrCat("Hello ", visitor, "! You are visitor #",
46                             ++visitor_count_, " to this enclave."));
47      }
48      return asylo::Status::OkStatus();
49    }
50
51   private:
52    uint64_t visitor_count_;
53  };
```

https://github.com/google/asylo/tree/master/asylo/examples/hello_world

Hello World in Asylo: Enclave Usage (part 1)

```
47  // Part 1: Initialization
48  asylo::EnclaveManager::Configure(asylo::EnclaveManagerOptions());
49  auto manager_result = asylo::EnclaveManager::Instance();
50  if (!manager_result.ok()) {
51    LOG(QFATAL) << "EnclaveManager unavailable: " << manager_result.status();
52  }
53  asylo::EnclaveManager *manager = manager_result.ValueOrDie();
54  std::cout << "Loading " << absl::GetFlag(FLAGS_enclave_path) << std::endl;
55
56  // Create an EnclaveLoadConfig object.
57  asylo::EnclaveLoadConfig load_config;
58  load_config.set_name("hello_enclave");
59
60  // Create an SgxLoadConfig object.
61  asylo::SgxLoadConfig sgx_config;
62  asylo::SgxLoadConfig::FileEnclaveConfig file_enclave_config;
63  file_enclave_config.set_enclave_path(absl::GetFlag(FLAGS_enclave_path));
64  *sgx_config.mutable_file_enclave_config() = file_enclave_config;
65  sgx_config.set_debug(true);
66
67  // Set an SGX message extension to load_config.
68  *load_config.MutableExtension(asylo::sgx_load_config) = sgx_config;
69  asylo::Status status = manager->LoadEnclave(load_config);
70  if (!status.ok()) {
71    LOG(QFATAL) << "Load " << absl::GetFlag(FLAGS_enclave_path)
72                << " failed: " << status;
73  }
74
```

https://github.com/google/asylo/tree/master/asylo/examples/hello_world

Hello World in Asylo: Enclave Usage (part 2)

```
// Part 2: Secure execution

asylo::EnclaveClient *client = manager->GetClient("hello_enclave");

for (const auto &name : names) {
  asylo::EnclaveInput input;
  input.MutableExtension(hello_world::enclave_input_hello)
      ->set_to_greet(name);

  asylo::EnclaveOutput output;
  status = client->EnterAndRun(input, &output);
  if (!status.ok()) {
    LOG(QFATAL) << "EnterAndRun failed: " << status;
  }

  if (!output.HasExtension(hello_world::enclave_output_hello)) {
    LOG(QFATAL) << "Enclave did not assign an ID for " << name;
  }

  std::cout << "Message from enclave: "
            << output.GetExtension(hello_world::enclave_output_hello)
                   .greeting_message()
            << std::endl;
}
```

https://github.com/google/asylo/tree/master/asylo/examples/hello_world

Hello World in Asylo: Enclave Usage (part 3)

```
// Part 3: Finalization

asylo::EnclaveFinal final_input;
status = manager->DestroyEnclave(client, final_input);
if (!status.ok()) {
  LOG(QFATAL) << "Destroy " << absl::GetFlag(FLAGS_enclave_path)
              << " failed: " << status;
}
```

https://github.com/google/asylo/tree/master/asylo/examples/hello_world

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

2. Port the smart meter class into Asylo framework fulfilling the following goals:
   - The counter value shall be protected all the time
   - Any functions that manipulate the counter value shall be protected against tampering attacks

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

3. Implement a client program that uses your protected SmartMeter class by:
   - Triggering the meterUsageCycle function; and
   - Displaying the counter value

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

Answer

- Demo!

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

4. Explain which attacks can*not* be implemented anymore, i.e., what enclave protection guarantees in the sample system.

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

4. Explain which attacks can*not* be implemented anymore, i.e., what enclave protection guarantees in the sample system.

Answer

In memory manipulation of the counter value is countered. The counter variable along with the sensitive functions (those that have access to the counter value) reside within the enclave and hence are protected against both data and code manipulations.

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

5. Enumerate and briefly discuss possible threats against your enclave-protected smart meter that (for example) jeopardize/undermine/manipulate the reading and represent the steps needed to carry out this attack using an ADT.

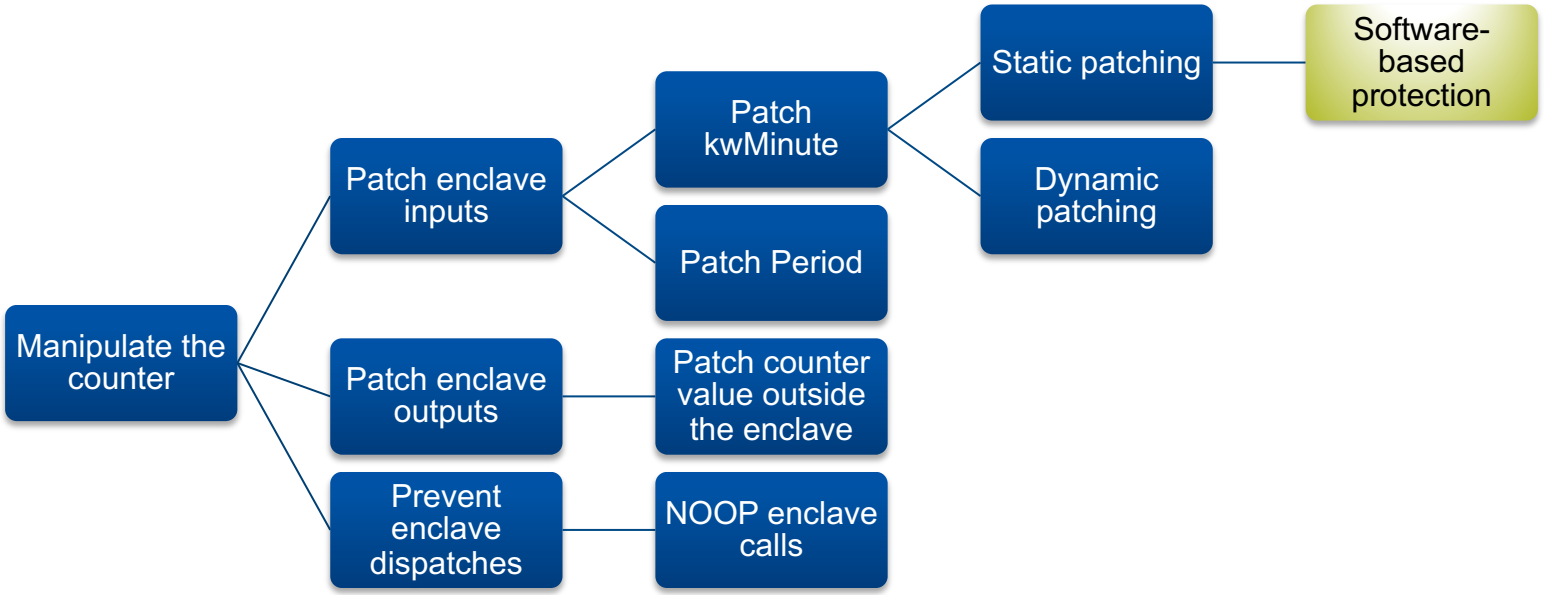
Answer

The interface to the enclave resides in the untrusted environment. The code residing outside the enclave is not protected against attacks. Despite the counter itself being protected within the enclave, the interface to functions that can manipulate the counter value resides outside the enclave. One possibility is to tamper with the input values of the enclave. The adversary can, for instance, change the kwMinute value outside the enclave. Similarly, perpetrators can patch the period to offPeak to get a 50% discount regardless of their usage time. Another possibility is to prevent the dispatch of usage reports to the enclave. Consequently, no usages will be counted by the smart counter. Lastly, the attackers can patch the output message of the enclave (in the untrusted world).

Exercise 6.2: Hardware-Based Protection in Practice (Hands-on)

5. Enumerate and briefly discuss possible threats against your enclave-protected smart meter that (for example) jeopardize/undermine/manipulate the reading and represent the steps needed to carry out this attack using an ADT.

Answer



Thank you

Any Questions?